

Assignment - I

1. Define algorithm efficiency
Algorithm efficiency refers to how well an algorithm performs in terms of
- ⊙ Time (how fast it runs)
 - ⊙ Space (how much memory it uses).

2. What are asymptotic notations

Asymptotic notations are used to describe the performance of an algorithm as the input size becomes very large

- Big-O Notation
- Omega Notation
- Theta Notation

3. Explain Time Complexity

Time complexity is the amount of time an algorithm takes to run as a function of input size.

4. Explain Big-O, Big-Theta and Big-Omega notations with ex
Big-O Notation

It represents the upper bound of an algorithm's running time. It tells us the maximum time an algorithm can take

If a function $f(n)$ grows no faster than $g(n)$, then:

$$f(n) = O(g(n))$$

Consider: $f(n) = 3n^2 + 5n + 2$

$$f(n) = O(n^2)$$

Linear Search in the worst case takes $O(n)$ time when the element is at the end or not present

Big Theta Notation
It represents the tight bound, meaning it gives both upper and lower bounds

If a function grows at the same rate as $g(n)$,
 $f(n) = \Theta(g(n))$

Traversing an array always takes $O(n)$ time

Big-omega Notation
It represents the lower bound of an algorithm i.e. the minimum time required

If a function grows at least as fast as $g(n)$,
 $f(n) = \Omega(g(n))$

Ex: in Linear Search
Best case: element found at first position $\rightarrow O(1)$

5. Describe the growth of functions in algorithm analysis

The growth of function refers to how the running time or space requirement increases as input size n increases

Help Constant notations
Predict performance for large inputs
Ignores constants and focuses on scalability

1. Constant Growth - $O(1)$
Time does not depend on input size
Ex - Accessing an array element

2. Logarithmic Growth - $O(\log n)$
Increases slowly
Ex - Binary Search

3. Linear Growth - $O(n)$
Time increases directly with input
Ex - Traversing an array

4. Linearithmic Growth - $O(n \log n)$
Slightly faster than quadratic
Ex - Merge Sort, Quick Sort

5. Quadratic Growth - $O(n^2)$
Time increases rapidly
Ex - Nested loops

6. Exponential Growth - $O(2^n)$
Very slow for large inputs
Ex - Recursive Fibonacci

1. Explain master theorem and its applications.
The Master Theorem is used to solve recurrence relations, especially in divide & conquer algorithms.

$$T(n) = aT(n/2) + f(n)$$

a = Subproblem no.
 $n/2$ = Size of each Subproblem
 $f(n)$ = Cost of dividing and combining

Three Cases of master Theorem:
① when $f(n)$ is Smaller
 $f(n) = O(n^{\log_2 a - \epsilon})$

② when $f(n)$ is equal
 $f(n) = O(n^{\log_2 a})$

③ when $f(n)$ is Larger
 $f(n) = O(n^{\log_2 a + \epsilon})$

Applications:-

- 1) Merge Sort $\rightarrow O(n \log n)$
- 2) Quick Sort (Average case) $\rightarrow O(n \log n)$
- 3) Binary Search $\rightarrow O(\log n)$
- 4) Strassen's matrix multiplication

2. Solve $T(n) = 2T(n/2) + n$ using master theorem.

Step 1: Identify values
 $a = 2$
 $b = 2$
 $f(n) = n$

Step 2: Compute
 $\log_2 a = \log_2 2 = 1$

Step 3: Compare $f(n)$

$$f(n) = n$$

$$n \log_2 a = n^1 = n$$

$$f(n) = O(n^{\log_2 a})$$

Step 4: Apply master Theorem
 $f(n) = O(n^{\log_2 a})$
 $\Rightarrow T(n) = O(n \log_2 n)$

Final answer

$$T(n) = O(n \log n)$$

2. Compare complexity of $n \log n$ and n^2 for $n = 100$

Step 1: Calculate values

① $n \log n$

(Take $\log$ base 2)

$$\log_2 100 \approx 6.64$$

$$n \log n = 100 \times 6.64 = 664$$

② n^2

$$n^2 = 100^2 = 10,000$$

Step 2: Compare

$$\frac{n \log n}{n^2} = \frac{664}{10,000}$$

$$n \log n \ll n^2$$

So, $n \log n$ is much more efficient than n^2 for large inputs

Assignment-2

1. Define divide and Conquer strategy.
Divide and Conquer is an algorithm design technique in which a problem is solved by breaking it into smaller sub problems & merge the solutions to get the final result.
2. What is Binary Search?
Binary Search is a searching algorithm used to find an element in a sorted array.
Time Complexity: $O(\log n)$
3. What is Merge Sort?
Merge Sort is a divide and Conquer Sorting algorithm.
 - a. Divide the array into two halves
 - b. Recursively Sort both halves
 - c. Merge the sorted halves
4. Explain quick Sort algorithm with example.
Quick Sort is a divide and Conquer Sorting algorithm that works by selecting a pivot element and partitioning the array around it.

Example

Array: [10, 7, 8, 9, 1, 5]

STEP 1: choose pivot (5)
Left: [1, 5]
Right: [10, 7, 8, 9]

STEP 2: Sort right part
Pivot = 9
Left: [7, 8]
Right: [10]

STEP 3: final Sorted array
[1, 5, 7, 8, 9, 10]

5. Describe Merge Sort and analyse its complexity.
Merge Sort is a divide & Conquer sorting algorithm that divides the array and merges sorted subarrays.

Working of Merge Sort

1. Divide the array into two halves
2. Recursively Sort each half
3. Merge the sorted halves

Complexity Analysis

Recurrence Relation:-

$$T(n) = 2T(n/2) + n$$

using Master Theorem

$$T(n) = O(n \log n)$$

- a. Explain Strassen's matrix multiplication algorithm.
 Strassen's algorithm is an efficient divide and conquer algorithm used for matrix multiplication. It reduces the number of multiplications instead of 8 (in recursive steps).

Given two matrices:-

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}, B = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$

$$\begin{aligned}
 M_1 &= (A_{11} + A_{22})(B_{11} + B_{22}) \\
 M_2 &= (A_{21} + A_{22})B_{11} \\
 M_3 &= A_{11}(B_{12} - B_{22}) \\
 M_4 &= A_{22}(B_{21} - B_{11}) \\
 M_5 &= (A_{11} + A_{12})B_{22} \\
 M_6 &= (A_{21} - A_{11})(B_{12} + B_{22}) \\
 M_7 &= (A_{12} - A_{22})(B_{21} + B_{22})
 \end{aligned}$$

7. Perform binary search on sorted array [5, 10, 15, 20, 25, 30] to find 20

Sorted Array: [5, 10, 15, 20, 25, 30]

Target = 20

Step 1) Initialize

low = 0

high = 5

Step 2) Find middle

$$\frac{0+5}{2} = 2$$

Since 20 > 15, Search in the right half

Step 3

low = 3

high = 5

$$\text{mid} = \frac{3+5}{2} = 4$$

Since 20 < 25, Search in the left half

Step 4

low = 3

high = 4

$$\text{mid} = \frac{3+4}{2} = 3$$

$$= 20$$

8. Determine recurrence relation of merge sort & solve it.
Merge sort divides the array into two halves and merges them.

$$T(n) = 2T\left(\frac{n}{2}\right) + n$$

time to sort two halves
time to merge

Solve using master theorem

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

$$a = 2$$

$$b = 2$$

$$f(n) = n$$

$$\log_b a = \log_2 2 = 1$$

$$\text{Compare } f(n)$$

$$f(n) = \Theta(n \log^0 n)$$

$$T(n) = \Theta(n \log^1 n)$$

Assignment - 3

1. What is greedy strategy?
The greedy strategy is an algorithmic approach where we make the best possible choice at each step, hoping it leads to the overall optimal solution.
2. Define minimum spanning tree.
A minimum spanning tree is a subset of edges in a connected weighted graph that
 → connects all vertices
 → has no cycles
 → has the minimum possible total edge-weight.
3. What is Huffman coding?
It is a greedy algorithm used for data compression.
 → Assign shorter codes to frequent characters
 → Assign longer codes to less frequent characters.
4. Explain Kruskal's algorithm ^{for MST}.
Kruskal's algorithm is a greedy algorithm used to find the minimum spanning tree of a connected weighted graph.

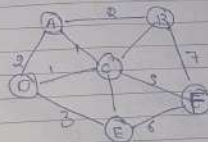
Steps of Kruskal's Algorithm

1. Sort all edges in increasing order of weight.
2. Start with an empty MST.
3. Pick the smallest edge.
4. Check:-
 • If it does not form a cycle, include it in MST.
 • Otherwise, discard it.

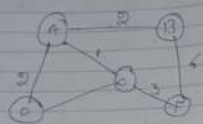
5. Repeat until MST has $(n-1)$ edges.

Time complexity $\Rightarrow O(n \log n)$

5. Describe Prim's algorithm with example.
Prim's algorithm is another greedy algorithm used to find MST. Start from any vertex. Grow the MST by adding the minimum weight edge connecting a new vertex.



Resultant Graph



1. Explain activity selection problem with greedy algo. It is a classic example of a greedy algorithm.

Problem Statement

→ Start time

→ Finish time

Select the maximum number of non-overlapping activities.

→ Always select the activity that finishes earliest.

Steps

1. Sort activities by finish time.

2. Select the first activity.

3. For remaining activities

→ Select the next activity whose start time $\geq$ last selected finish time.

4. Repeat

Time Complexity

→ Sorting : $O(n \log n)$

→ Selection : $O(n)$

Applications

- Scheduling problems
- Resource allocation
- CPU task scheduling

1. Arrange frequencies in Arrange frequencies
5, 9, 12, 13, 16, 45

2. Build Huffman Tree

Iteration 1:

Pick smallest 5 and 9

Combine 14

New list: 12, 13, 16, 45

Iteration 2:

Pick 12 and 13

Combine 25

New list: 14, 16, 25, 45

Iteration 3:

Pick 14 and 16

Combine 30

→ 25, 30, 45

Iteration 4:

Pick 25 and 30

Combine 55

45, 55

Iterations
 0.5 x 100
 53.100

Iteration codes
 100.100.100.1

Frequency	Code
43	0
16	11.1
13	10.1
12	10.0
9	11.0.1
5	11.0.0

Final Huffman codes

Frequency	Huffman Codes
5	11.0.0
9	11.0.1
12	10.0
13	10.1
16	11.1
43	0

Imagine graph



	A	B	C	D
A	0	1	4	0.0
B	0	1	3	6
C	0	1	3	4

A → A → 0
 A → B → 1
 A → C → 3
 A → D → 4

Assignment-4

1. Define dynamic programming.
Dynamic programming is an algorithm design technique used to solve problems by breaking them into smaller subproblems, solving each subproblem once, and storing the results for future use.
2. What is optimal substructure?
A problem has optimal substructure if an optimal solution to the problem can be constructed from optimal solutions of its subproblem.
3. Define overlapping subproblems.
If the same subproblems are solved multiple times during computation.
4. Explain matrix chain multiplication problem.
Matrix chain multiplication is a dynamic programming problem that determines the most efficient way to multiply a chain of matrices.
 $A_1, A_2, A_3, \dots, A_n$

Dynamic Programming approach Recursive Relation

1. Steps
 2. Define problem's subproblems.
 3. Store results in a table.
 4. Build solution bottom-up.
- Time Complexity = $O(n^3)$

Applications
→ Compiler optimization
→ Graphs' Computations
→ Scientific Computing

5. Describe Floyd-Warshall algorithm.
Floyd-Warshall is a dynamic programming algorithm used to find the shortest paths between all pairs of vertices in a graph.

→ works for weighted graphs
→ Handles positive and negative weights

- Steps
1. Initialize distance matrix
 2. for each intermediate vertex k :
• update all pairs (i, j)
 3. Repeat for all vertices

Time Complexity = $O(n^3)$

6. Explain Longest Common Subsequence Problem
The longest common subsequence problem finds the longest sequence that appears in the same order in two strings.

Problem Statement

$x = x_1, x_2, \dots, x_m$
 $y = y_1, y_2, \dots, y_n$

Recurrence Relation

$$L[i][j] = \begin{cases} 0 & \text{if } i=0 \text{ or } j=0 \\ \max(L[i-1][j], L[i][j-1]) & \text{if } x[i] \neq y[j] \\ L[i-1][j-1] + 1 & \text{if } x[i] = y[j] \end{cases}$$

7. Solve rod cutting problem for price list $[2, 5, 8, 9]$
- Steps
1. Create a table of size $(m+1) \times (n+1)$
 2. Fill it using recurrence
 3. Trace back to find the subsequence

Given $m=4$

length 1 $\rightarrow 2$
length 2 $\rightarrow 5$
length 3 $\rightarrow 8$
length 4 $\rightarrow 9$

Step 1: Recurrence Relation

$$R(n) = \max_{1 \leq k \leq n} (P_k + R(n-k))$$

Step 2: Compute values

for $n=1$

$$R(1) = 2$$

for $n=2$

Options:

$\rightarrow$ cut $(1,1) \rightarrow 2+2=4$

$\rightarrow$ No cut $\rightarrow 5$

$$R(2) = 5$$

for $n=3$

Options:

$(1,2) \rightarrow 2+3=5$

$(2,1) \rightarrow 5+2=7$

No cut $\rightarrow 8$

$$R(3) = 8$$

for $n=4$

$(1,3) \rightarrow 2+8=10$

$(2,2) \rightarrow 5+5=10$

$(3,1) \rightarrow 8+2=10$

No cut $\rightarrow 9$

$$R(4) = 10$$

8. Compute LCS length for strings 'ABCBDAB' & 'BACBBD'

$X = ABCBDAB$

$Y = BACBBD$

Step 1: Use LCS formula.

$$L[i][j] = \begin{cases} 0 & \text{if } i=0 \text{ or } j=0 \\ \max(L[i-1][j], L[i][j-1]) & \text{if } X[i] \neq Y[j] \\ L[i-1][j-1] + 1 & \text{if } X[i] = Y[j] \end{cases}$$

	B	D	C	A	B	A
A	0	0	0	1	1	2
B	1	1	1	1	2	3
C	1	1	2	2	2	3
D	1	1	2	2	3	3
A	1	2	2	3	3	4
B	1	2	2	3	4	4

LC length = 4

are possible LCS :-

5182

170912

Red cutting $\rightarrow$ Max Profit = 10

LC $\rightarrow$ length = 4

Assignment - 5

1. Define Branch and Bound technique
Branch and bound is an algorithm design technique used to solve optimization problems.

Branch $\Rightarrow$ Divide the problem
Bound $\Rightarrow$ calculate a bound to decide whether a subproblem is worth exploring

2. what is LC Branch and Bound.
LC Branch and Bound is a variation where the next node to explore is chosen based on the least cost.
3. what is Bounding function?
A bounding function is used to estimate the best possible solution from a given node.
4. what is FIFO Branch & Bound
FIFO Branch and Bound is a strategy where nodes are explored in the order they are generated, using a queue.

STEPS

1. Start with the root node and insert it into a queue.
2. Remove the front node from the queue.
3. Generate its child nodes.

4. Compute bounds for each child
5. If a child is promising, add it to the queue
6. Repeat until the optimal solution is found

Example:

- In a problem like 0/1 knapsack
- generate all possible combinations level by level
 - discard nodes that do not satisfy bounding conditions

5. Discuss Branch and Bound for 0/1 knapsack

The 0/1 knapsack problem involves selecting items to maximize profit without exceeding capacity

- Explore subsets of items
- use a bounding function to eliminate non-promising solutions

Steps

1. Sort items
2. Create state space tree
3. Bounding function
4. Branching

Item	Weight	Profit
1	2	40
2	3	30
3	5	100

Capacity = 5

Best choice: Item 1, Item 2
Total weight = 5
Profit = 70

Time complexity
Worst case: $O(2^n)$

Feature	Backtracking	Branch and Bound
1. Goal	Find all solutions	Find optimal solution
2. Approach	Depth-first Search	BFS
3. Pruning	Based on constraints	Based on bounds (cost)
4. Optimality	No necessarily optimal	Always optimal
5. Search Strategy	Recursive	uses queue
6. Examples	N-Queens, Sudoku	knapsack, TSP

7. Solve small 0/1 knapsack instance using Branch and Bound

Given problem

Item	Weight	Profit
1	2	40
2	3	30
3	5	100

Capacity = 5

Step 1: Item	Profit	Weight
1	60	10
2	100	20
3	120	30

Step 2: State Space Tree Concept
Level, Profit and Weight Each node

Contains
Profit = 0
Weight = 0

Step 3: Branching

Include Item 1
Weight = 10
Profit = 60

Exclude Item 1
Weight = 0
Profit = 0

Step 4: Include Item 2
Weight = 10 + 20 = 30
Profit = 60 + 100 = 160

Include Item 3
Weight = 30 + 30 = 60 (Exceeds Capacity)

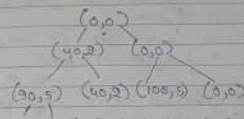
Step 5:

Combination	Weight	Profit
1 + 2	30	160
3	30	120

Optimal Profit = 160
Select Item 1 & 2 only

2. Illustrate state space tree for knapsack problem
A state space tree represents all possible solutions

Each level $\rightarrow$ decision about one item
Left branch $\rightarrow$ include item
Right branch $\rightarrow$ exclude item



Assignment-8

1. Define NP-Complete problems.
NP-Complete problems are a class of problems in Computational Complexity theory that are:
→ their solutions can be verified in polynomial time
→ Every problem in NP can be reduced to them in polynomial time
2. What is polynomial time complexity?
Polynomial time complexity refers to algorithms whose running time can be expressed as a polynomial function of input size n .
3. What is reduction in Complexity theory?
Reduction is a technique used to transform one problem into another problem.
4. Explain NP-Hard and NP-Complete classes.
NP (Nondeterministic Polynomial Time)
Problems whose solutions can be verified in polynomial time
→ Every problem in NP can be reduced to it in polynomial time

Key features:

- may or may not be in NP
 - may not even have a verifiable solution in polynomial time.
 - At least as hard as the hardest problem in NP
- Example - Traveling Salesman Problem, Halting Problem

NP-Complete Problems

- It is in NP, and
- It is NP-Hard
- Hardest problem within NP
- If one NP-complete problem is solved in polynomial time $\rightarrow P = NP$

$$EX = 3-SAT$$

Vertex cover

5. Describe Vertex Cover and Subset Sum problems.
Vertex cover problem
Given a graph $G(V, E)$, a vertex cover is a set of vertices such that:
→ Every edge has at least one endpoint in the set

Examples

Graph:

Edges: $(a,b); (b,c); (c,d)$

Possible vertex covers

- $\{b\}$ (minimum)
- $\{a, c\}$

- Subset Sum problem
Given a set of integers and a target sum:
Determine if there exists a subset whose sum equals the target

E.g. $\rightarrow$ set: $\{3, 34, 4, 12, 5, 8\}$
Target = 3

Solution
 $\{3, 5, 3\} \rightarrow \text{Sum} = 3$

Applications
Cryptography
Partition problems

6. Discuss approximation algorithms
Approximation Algorithms
They are used to find near-optimal solutions for problems where finding the exact solution is difficult

Need for Approximation

- $\rightarrow$ Exact Algorithms take exponential time
- $\rightarrow$ Not practical for large inputs

They Run in polynomial time and produces a solution close to the optimal

Approximation Ratio

$\frac{\text{Approximate Solution}}{\text{Optimal Solution}}$

7. Determine complexity class of Traveling Salesman problem
Given a set of cities and distances between them, find the shortest path or possible route that visits each city once and returns to starting city

Two versions of TSP

1. Decision Version (NP-complete)
2. Optimization version (NP-hard)

NP-complete

A given solution can be verified in polynomial time

NP-hard

Finding the best tour is harder than verifying

8. Analyze time complexity of string matching algorithm
It contains pattern with every substring of text

Time complexity

Best Case $\propto O(n)$

Worst Case $\propto O(n^2)$

$n = \text{length of text}$
 $m = \text{length of pattern}$